

Oracle-Anchored LVR Recapture with Selective Dutch Auction Repricing

Mitchell Marfinetz
University of Pennsylvania
`mitchmar@sas.upenn.edu`

August 2026

Abstract

Liquidity providers in automated market makers lose value when a pool price is stale relative to a faster reference market. We study a Uniswap v4 hook that detects stale pool states with an external oracle, charges toxic repricing flow with a square-root fee rule derived from one-step loss-versus-rebalancing (LVR), and applies a permissionless Dutch-auction concession. On an observed-flow replay covering 54 windows across seven Uniswap v3 pools, the selective policy improves LP uplift versus a broad all-stale auction in 19 windows, leaves 35 unchanged, and worsens 0. Its mean window trigger rate is 3.07%, versus 7.17% for the broad policy. Volatility is measurable in 38 windows, all classified normal; 16 are excluded rather than assigned a default regime. A separate October 2025 corpus contains 95 measured normal and 29 measured stress windows at a 100% annualised-volatility threshold. Because recapture is bounded by oracle freshness, we also compare with a faster CEX reference: mainnet Chainlink lags Binance by a median 21 bps, limiting recapture of CEX-measured LVR even when oracle-visible stale loss is nearly recovered.

1 Introduction

An automated market maker (AMM) quotes prices from its own on-chain state. If an external market moves first, the AMM can briefly quote a stale price. A fast arbitrageur can then trade against the stale pool, move the pool toward the external price, and capture value that would otherwise belong to liquidity providers (LPs). This paper studies that event at hook level: when a Uniswap v4 pool is stale, can a hook redirect stale-price value back to LPs without leaving the pool unrepriced?

The accounting object is loss-versus-rebalancing (LVR) [1]. LVR measures the value LPs lose relative to a continuously rebalanced inventory benchmark when they provide liquidity to an AMM. In this paper, we use LVR in a narrower execution sense: a one-step stale repricing trade creates a measurable stale-price loss for LPs before fees. A hook can then decide whether to charge the repricing trader directly, open a solver auction around that repricing right, or leave ordinary AMM execution unchanged.

The relevant design question is not whether this mechanism beats an old static-fee pool. Fixed-fee Uniswap v3 is useful as a legacy leakage reference, but it is not the right primary benchmark for a v4 hook. The real benchmark is a set of v4-hook alternatives:

- **Base oracle-anchored hook:** detects stale prices and charges the deterministic square-root hook fee, but has no Dutch auction path.

- **Broad all-stale auction hook:** opens auctions for every stale/toxic opportunity after a solver-cost reserve.
- **Selective auction hook:** opens an auction only when it improves LP accounting relative to the base hook fee.

The contribution is therefore threefold. First, we give a plain hook-level interpretation of stale-price LVR and derive the square-root fee that recovers the one-step stale loss in a constant-product pool. Second, we position the design against existing v4 hook patterns: dynamic-fee hooks, oracle hooks, and auction-managed AMM work. Third, we evaluate the incremental auction design against hook-level baselines, showing that the selective rule triggers less often than the broad auction rule while improving LP uplift relative to the base hook.

2 Reader Guide and Definitions

This section defines the terms used throughout the paper before introducing the formulas.

AMM.

An automated market maker is an on-chain exchange where smart-contract state determines the swap price. Uniswap v3 and v4 pools are AMMs.

LP. A liquidity provider deposits assets into a pool. In return, the LP earns fees but bears inventory and adverse-selection risk.

Hook.

In Uniswap v4, a hook is an external smart contract attached to a pool. It can run before or after pool actions such as swaps and liquidity changes. Hooks are the v4 mechanism for pool-specific custom logic.

Dynamic fee.

A v4 dynamic fee is a swap fee that can change over time or per swap. A `beforeSwap` hook can return a fee override in a dynamic-fee pool.

Oracle.

An oracle is a reference price source. This paper uses Chainlink as the primary reference and treats Pyth, Binance, and deep-pool references as sensitivity inputs.

Stale price.

A pool is stale when its current pool price differs materially from the reference price. The paper measures this through a pool-oracle stale gap.

LVR.

Loss-versus-rebalancing is the adverse-selection cost LPs pay when AMM inventory is rebalanced at stale prices rather than continuously marked to a reference market.

Toxic flow.

In this paper, toxic flow means informed stale-price repricing flow. It is toxic because it trades against a stale quote and creates LP loss before fees.

Solver.

A solver is an actor that can fill a repricing auction. The solver receives a concession from the stale-price value and LPs receive the rest.

Dutch auction.

A Dutch auction starts with a low solver concession and increases it over time until a solver clears, the auction expires, or a reserve/cap blocks execution.

LP net.

LP net is fee revenue minus gross stale LVR. Positive LP net means fees more than cover the modeled stale-price loss.

LP uplift vs base hook.

LP uplift versus the base hook is the incremental LP net created by an auction rule relative to the deterministic hook-only fee path.

3 Related Work and Positioning

This paper piggybacks on three existing strands of work.

First, the LVR literature gives the economic accounting. Milionis, Moallemi, Roughgarden, and Zhang define LVR as an AMM adverse-selection cost relative to a continuously rebalanced benchmark [1]. We do not build a continuous-time AMM model. We use the LVR framing to price a single stale repricing event and then ask how a v4 hook should respond to that event.

Second, Uniswap v3 and v4 provide the market structure. Uniswap v3 introduced concentrated liquidity, which makes liquidity more capital efficient but also makes position shape important [2]. Uniswap v4 keeps concentrated liquidity and adds hooks, singleton accounting, and more flexible fee logic [3]. Official v4 documentation describes hooks as pool-attached contracts that can customize behavior around swap and liquidity lifecycle points [4], and describes dynamic fees as a v4 mechanism that can adjust fees per swap through `beforeSwap` [5]. Our hook is therefore not a new AMM; it is a v4 customization of swap pricing and liquidity admission.

Third, auction and hook designs provide the comparison class. The am-AMM paper studies auction-managed AMMs where the auction sells broader AMM control rights [8]. Frequent batch auction work motivates auctions as a response to speed races [9]. Existing v4 hook examples show that hooks can implement specialized execution and oracle logic: TWAMM hooks split large orders over time [6], truncated oracle hooks smooth oracle movements [7], and dynamic-fee hooks adjust swap fees [5]. Our mechanism is narrower than am-AMM governance and narrower than generic dynamic fees. It sells a short-lived stale-repricing right only when the hook counterfactual leaves LPs worse off.

Design family	What it controls	Difference from this paper
Fixed-fee AMM	Static fee tier	Legacy reference only; cannot distinguish stale repricing flow from ordinary flow.
Generic dynamic-fee hook	Fee level as a function of volatility, volume, source, or other state	Our fee is tied to one-step stale-loss accounting, not a heuristic.
Oracle hook	Reference-price or oracle update behavior	Our oracle gap triggers repricing protection rather than publishing an oracle alone.
TWAMM hook	Time-sliced large-order execution	Our auction concerns stale repricing rights, not long-horizon user order slicing.
am-AMM / auction-managed AMM	Broader auctioned AMM control rights	Our auction is local and conditional: one stale state, one short-lived repricing opportunity.

4 Mechanism Overview

The proposed hook has three pieces.

The first piece is the **base hook fee**. When the oracle and pool disagree, the hook computes an exact premium from the square-root ratio between the oracle price and the pool price. The deterministic fee is

$$\text{hookFee} = \text{baseFee} + \alpha \cdot \text{exactPremium},$$

subject to **maxFee**. This is the hook-only counterfactual used as the main benchmark.

The second piece is the **stale-gap trigger**. The unsigned gap is

$$\text{stale_gap_bps_before} = |\log(\text{oracle_mid}/\text{pool_mid})| \times 10,000.$$

An auction can open only when

$$\text{stale_gap_bps_before} \geq \text{trigger_gap_bps}.$$

The sign is stored separately. A positive sign means the oracle is above the pool, so the pool is stale low.

The third piece is the **selective Dutch auction**. The broad auction rule opens too often because it auctions any stale/toxic opportunity after a solver-cost reserve. The selective rule instead asks whether the auction improves LP net relative to the base hook. If the base hook already protects LPs, the auction does not open. If the base hook leaves LPs exposed, the auction starts with a small solver concession and increases that concession until a solver clears or the reserve/cap blocks the fill.

5 Fee Formula

The square-root fee is an accounting reserve for stale repricing, not an arbitrary fee knob. Let the pool mid be P and the reference mid be $P' = Pe^z$, where z is the signed log gap. For a

constant-product pool with invariant $xy = L^2$, the pool state aligned with price P is

$$x(P) = \frac{L}{\sqrt{P}}, \quad y(P) = L\sqrt{P}.$$

For a one-step repricing from P to P' , the exact fee fraction on the informed input that recovers stale-price loss is

$$f^*(z) = e^{|z|/2} - 1.$$

5.1 Intuitive geometric-mean proof

The easiest way to see the formula is through the AMM's average execution price. A constant-product pool does not reprice from P to P' at either endpoint price. The average execution price over the full repricing trade is the geometric mean:

$$\bar{P} = \sqrt{PP'}.$$

This follows directly from the reserve changes. If the pool starts at price $P = y/x$ and ends at P' , the quote paid divided by base received over the constant-product path equals $\sqrt{PP'}$.

Now consider $P' > P$, meaning the oracle is above the pool and the pool is stale low. The informed trader buys base from the pool at average price $\bar{P}$, while that base is worth P' at the reference market. The stale value extracted per unit of quote paid is therefore

$$\frac{P' - \bar{P}}{\bar{P}} = \frac{P'}{\sqrt{PP'}} - 1 = \sqrt{\frac{P'}{P}} - 1.$$

This is the required fee fraction on the quote input if the hook wants to recover the one-step stale loss.

The other direction is symmetric. If $P' < P$, the pool is stale high. The informed trader sells base into the pool at average price $\bar{P}$, while that base is worth only P' at the reference market. The stale value extracted per unit of base input, valued at P' , is

$$\frac{\bar{P} - P'}{P'} = \frac{\sqrt{PP'}}{P'} - 1 = \sqrt{\frac{P}{P'}} - 1.$$

Both directions therefore reduce to

$$f^* = \sqrt{\frac{\max(P, P')}{\min(P, P')}} - 1 = e^{|z|/2} - 1,$$

where $z = \log(P'/P)$. Locally,

$$f^*(z) = \frac{|z|}{2} + O(z^2),$$

so the familiar half-gap rule is the first-order approximation.

Written in oracle and pool mids, the exact premium is

$$\text{exactPremium} = \begin{cases} \sqrt{\text{oracle.mid}/\text{pool.mid}} - 1, & \text{oracle above pool,} \\ \sqrt{\text{pool.mid}/\text{oracle.mid}} - 1, & \text{oracle below pool.} \end{cases}$$

The Dutch auction does not replace this formula. It uses the base-hook fee path as the reserve/counterfactual that the auction must improve.

6 Liquidity-Width Guard

A swap fee controls what happens when stale flow arrives. It does not control how much fragile liquidity is exposed before the stale flow arrives. The implemented hook therefore also has a liquidity-width and centering guard for incoming LP positions.

The reason is specific to concentrated liquidity. Narrower ranges can multiply stale loss per dollar of position value. For a centered range $[P_a, P_b]$, define total log width $W = \log(P_b/P_a)$ and center $P_0 = \sqrt{P_a P_b}$. At the center, the position value is

$$V(P_0) = 2L\sqrt{P_0}(1 - e^{-W/4}).$$

The normalized gross-LVR rate at the center is amplified to

$$\frac{\ell(\sigma, P_0)}{V(P_0)} = \frac{\sigma^2}{8(1 - e^{-W/4})}.$$

If the designer budgets gross LVR over an effective latency window Δ at no more than ϵ of position value, it suffices to require

$$\frac{\sigma^2 \Delta}{8(1 - e^{-W/4})} \leq \epsilon, \quad W \geq W_{\min} = -4 \log \left(1 - \frac{\sigma^2 \Delta}{8\epsilon} \right).$$

The October auction grid does not empirically validate LP repositioning behavior. The guard is included to show that the hook design addresses both sides of the stale-flow problem: `beforeSwap` prices stale repricing flow, while `beforeAddLiquidity` rejects marginal ranges that exceed the configured LVR budget.

7 Empirical Design

The evidence has two layers.

7.1 Data and Replay Construction

The historical input is Ethereum mainnet swap and `Sync` event data from Uniswap v3 pools. The replay reconstructs v3 pool state block by block, classifies each swap as informed or uninformed under the stale-gap rule defined in Section 3, and then re-prices the same observed flow through (a) the unmodified v3 fee tier, (b) the base oracle-anchored hook, (c) the broad all-stale auction overlay, and (d) the selective auction overlay. Reference prices come from Chainlink USD and ETH feeds for the relevant tokens; Pyth, Binance, and deep-pool references are reserved for sensitivity checks and are not used in the headline numbers.

The observed-flow replay covers 54 windows across seven Uniswap v3 pools, totalling roughly 180 hours of trading. A **window** is a contiguous block range over which informed/uninformed classification, fee accounting, and auction overlays are evaluated on the same swap stream. The source manifests retain their original routine/stress sampling tags for provenance, but those tags are not observations. Regime is instead computed from annualised realized volatility of the primary reference series. Only 38 prefixes contain enough primary-feed observations; all are measured normal at the 100% threshold, while 16 remain unclassified. The observed-flow ablation therefore does not establish stress-regime generalisation. The October 2025 calibration grid is the separate regime check: it covers WETH/USDC, WBTC/USDC, LINK/WETH, and UNI/WETH across 124 four-hour windows (31 per pool), 89,220 swap samples, and 8,784 oracle updates, split into 95 measured normal and 29 measured stress windows.

7.2 Primary Benchmark: V4 Hook Ablation

The primary benchmark is the observed-flow replay. It reconstructs v3 pool state, classifies observed swaps as informed or uninformed under the replay labels, replays fixed, hook, linear, and log fee policies, and overlays two auction policies on the same 54 windows. Declared sampling labels are retained but never substituted for missing measured volatility.

The comparison is deliberately hook-native:

Benchmark	Trigger/reserve rule	Interpretation
Base hook	No auction path	Oracle-anchored square-root fee only. This is the hook-without-auction counterfactual.
Broad all-stale auction	<code>all_toxic</code> trigger, solver-cost reserve	Earlier/simple auction policy. It auctions more stale events.
Selective auction hook	<code>auction_beats_hook</code> trigger, <code>hook_counterfactual</code> reserve	Proposed policy. It auctions only when LP accounting improves relative to the base hook.

The reported LP columns are in each window’s native quote units and should not be summed as one cross-pool dollar P&L. The sign and direction are the relevant evidence.

7.3 Parameter Calibration: October 2025 Grid

The second layer is an October 2025 grid for four pools. It uses simulated correction trades on stale repricing opportunities and sweeps:

Parameter	Values
<code>trigger_gap_bps</code>	5, 10, 25, 50
<code>base_fee_bps</code>	1, 5, 30
<code>start_concession_bps</code>	10, 30, 100
<code>concession_growth_bps_per_sec</code>	0.5, 1.0, 5.0
<code>max_fee_bps</code>	500, 2500, 5000

This grid has 1,296 pool-parameter rows and 40,176 window-parameter rows. It is used to pick an auction-layer candidate around the base hook fee law, not to claim final production market share.

7.4 Legacy Reference: Fixed-Fee V3

Fixed-fee v3 remains in the paper only as a legacy leakage check. It asks how much stale-price loss an ordinary production fee tier would recover from the same modeled stale repricing opportunities. That is useful context, but it is not the benchmark that establishes the incremental value of the proposed v4 hook design.

8 Results

8.1 Hook-Level Benchmark Results

The main result is that the selective auction improves the broad auction rule while preserving the base hook as a reserve. Across the 54-window observed-flow replay, the selective rule improves LP uplift versus the broad all-stale auction in 19 windows, leaves 35 unchanged, and worsens 0.

Subset	N	Improved	Broad	Selective	Delta	95% CI
Overall	54	19	0.3609	1.3501	0.9892	[0.0065, 2.2198]
Measured normal	38	14	0.5115	1.7088	1.1973	[0.0145, 2.8729]

The unmeasurable subset has 16 windows: 5 improve, 11 are unchanged, and 0 worsen. These rows remain in the overall policy comparison but are excluded from every regime claim.

The selective rule is also materially less intrusive. The broad all-stale rule triggers on 7.17% of observed replay opportunities on average by window. The selective rule triggers on 3.07%. Event-weighted across 7,106 swaps, the selective arm triggers 130 auctions and fills 119, a 91.54% fill rate; 11 fall back when the oracle is stale at modeled fill time. The model therefore supports reduced intervention, not perfect live liveness.

This is the core benchmark result. The proposed design does not merely add an auction to the hook. It adds a more selective auction gate that preserves the base hook counterfactual and avoids auctioning stale events where the deterministic hook fee already protects LPs.

8.2 Auction Parameter Calibration

The selected October 2025 parameter set is:

$$(10, 5, 10, 0.5, 2500),$$

in the order trigger gap, base fee, starting concession, concession growth, and max fee. The choice is not an unconstrained optimum. It is a conservative deployment candidate within the tested grid: 10 bps trigger gap, 5 bps base fee, 10 bps starting solver concession, 0.5 bps/sec concession growth, and 2500 bps max fee.

Pool	Recapture	Clear rate	Legacy v3 ref.	Triggers	Solver payout
WETH/USDC	99.9000%	1.0000	52.5709%	1242	10.0000 bps
WBTC/USDC	99.9000%	1.0000	16.9001%	800	9.9995 bps
LINK/WETH	99.9000%	1.0000	24.4175%	3163	10.0000 bps
UNI/WETH	99.9000%	1.0000	37.7356%	2209	10.0000 bps

The uniform 99.9% recapture is a mechanical consequence of the selected auction concession, not a claim that all four pools have identical economics. The starting concession gives the solver 10 bps of gross stale-LVR value, not 10 bps of trade notional, so a quickly clearing auction leaves roughly 99.9% of modeled stale LVR to LPs.

8.3 The oracle-freshness ceiling on recapture

The 99.9% above is recapture of the *oracle-visible* stale-loss: LVR is measured against the same Chainlink series the hook uses as its oracle, so the mechanism cannot, by construction, miss a gap its own oracle does not see. A real arbitrageur trades against the continuously-moving CEX price, whereas a Chainlink feed updates only on a deviation threshold or heartbeat. To measure true recapture we re-price every executed swap’s stale-loss against a faster Binance 1-minute reference while keeping Chainlink as the hook oracle, over 24 stratified 2026 windows on WETH/USDC (mainnet feeds), using the same `correction.trade` LVR function. The Chainlink feed lags the CEX by a median of 21 bps at swap times (p90 $\approx$ 47–64 bps), so only $\approx$ 78% of true CEX-measured LVR is oracle-visible, and true recapture is about 78% rather than 99.9%. The visible fraction

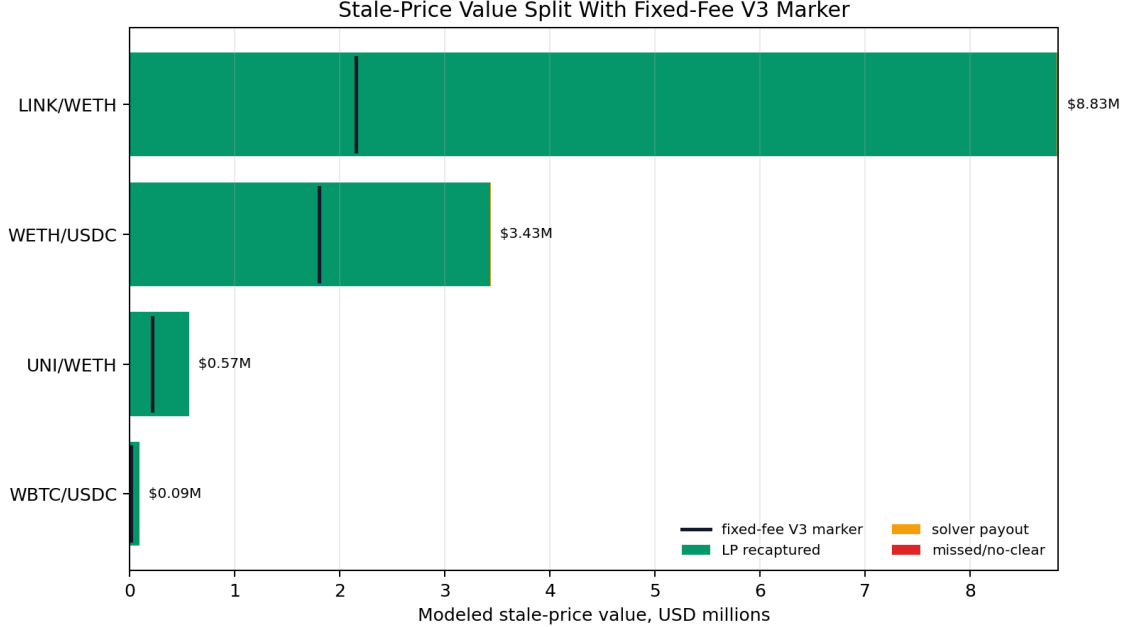


Figure 1: October 2025 calibration: USD-equivalent stale-price value split for the selected parameter set by pool. Green bars show LP-recovered stale value, orange shows solver payout, red shows missed/no-clear value, and the black marker shows the fixed-fee v3 legacy reference on the same modeled opportunities.

falls to $\approx 40\text{--}45\%$ in the highest-volatility months and is $\approx 85\%$ in the calmest: the mechanism recaptures a smaller share of true LVR exactly when LVR is largest, because the CEX moves fastest then and the deviation-triggered feed lags most. This is a ceiling on *any* oracle-anchored recapture mechanism, set by oracle freshness: the tested mainnet feeds carry a 0.5% deviation threshold, whereas the Base ETH/USD feed is 0.15% ($\approx 3\times$ tighter), so a Base deployment’s true recapture is materially higher; a low-latency reference (Chainlink Data Streams or an aggregated oracle) behind the same interface closes the remaining gap without changing the fee law or auction.

The solver economics are the main caveat:

Pool	Filled auctions	Stale value	Solver payout	Avg payout
LINK/WETH	3163	\$8.83M	\$8.8k	\$2.79
WETH/USDC	1242	\$3.43M	\$3.4k	\$2.77
UNI/WETH	2209	\$565.1k	\$565.12	\$0.26
WBTC/USDC	800	\$92.7k	\$92.72	\$0.12
Total	7414	\$12.93M	\$12.9k	\$1.74

The average payout is a break-even cost threshold. A solver must clear the auction below that cost, including gas and operational overhead, before the fill is profitable. This makes low-cost execution, batching, and multi-pool correction important future design requirements.

Two properties of this payout are easy to over-read. First, it is heavily volatility-dependent and fat-tailed. October 2025 contains a market dislocation; a six-month 2026 re-run on WETH/USDC (24 stratified windows) gives a *median* per-fill payout near \$0.003 with a mean near \$0.05, because the concession is $\approx 0.1\%$ of each fill’s stale-loss and that scales with the gap. The per-fill payout

should be read as a distribution, not a single mean. Second, the payout is the *residual* left to the filler after LPs keep $\approx 99.9\%$; it is not the filler’s total incentive. In a zero-gas backtest the auction clears at a near-zero concession because filling is free; with real gas the concession clock grows until a fill clears gas plus a margin, so a rational filler always earns at least their gas back by construction. A gas-aware re-tune (Base gas, `solver_cost` fill gate) confirms fill rate stays $\approx 99.7\%$ and shows the concession *growth* rate is second-order while the *start* concession is the effective lever trading LP recapture against filler incentive.

8.4 Sensitivity

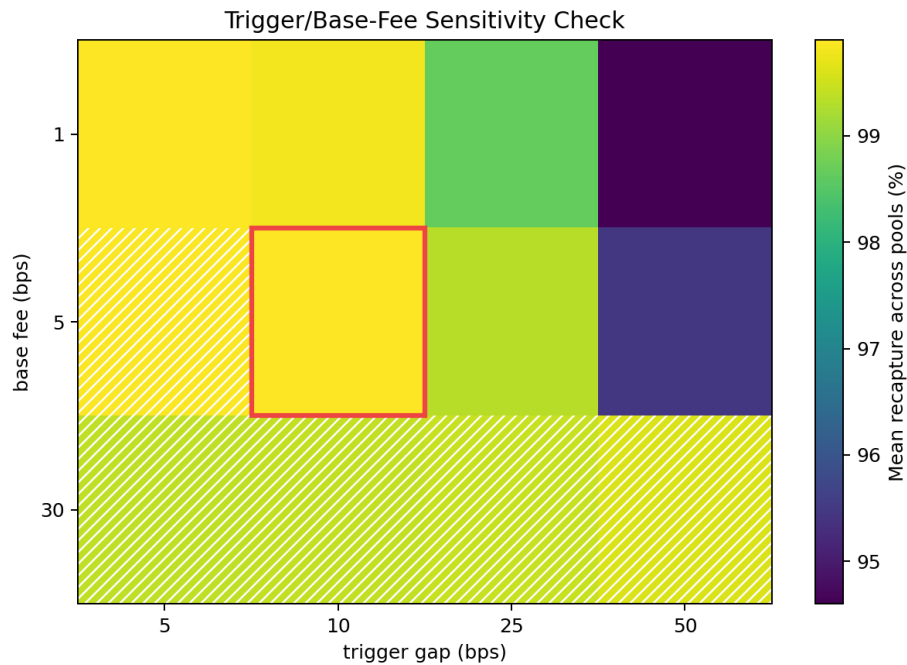


Figure 2: Trigger-gap by base-fee sensitivity check. The other auction parameters are fixed at the recommended values; hatched entries have at least one pool below 0.9 clear rate, and the red outline marks the recommended parameter set.

The largest one-step effect in the October grid is `base_fee_bps`: increasing the base fee lowers measured recapture because it removes marginal clears. The next-largest effects are starting concession and trigger gap. `max_fee_bps` and concession growth are second-order over this month slice.

Parameter step	Mean delta	Std delta	Pools improved
<code>base_fee_bps</code> up	-0.986049 pp	0.562620 pp	0/4
<code>start_concession_bps</code> up	-0.699985 pp	0.000013 pp	0/4
<code>trigger_gap_bps</code> up	-0.576773 pp	0.424254 pp	0/4
<code>start_concession_bps</code> down	0.199997 pp	0.000004 pp	4/4

8.5 Legacy V3 Leakage Check

The v3 fee tier is not the primary benchmark, but it remains useful to show the leakage that motivates a v4 mechanism. A static fee collects a percentage of trade notional; stale LVR is measured against the stale repricing opportunity. The two are not the same object.

Pool	V3 fee	V3 recapture	Implied stale loss	Gap to selected rule
WETH/USDC	30 bps	52.5709%	57.07 bps	47.3291 pp
WBTC/USDC	5 bps	16.9001%	29.59 bps	82.9999 pp
LINK/WETH	30 bps	24.4175%	122.86 bps	75.4825 pp
UNI/WETH	30 bps	37.7356%	79.50 bps	62.1644 pp

This table should be read as motivation, not as proof of superiority over v4 hook designs. The proof-of-design comparison is the hook ablation above.

9 Discussion

The revised evidence supports a more precise story than the earlier draft. The base oracle-anchored hook supplies the stale-loss fee floor. The broad auction rule shows what happens when auction logic is added too aggressively. The selective Dutch auction is better because it treats the base hook as a counterfactual reserve: it acts only where the auction can improve LP accounting.

This matters for an external audience because the mechanism has two goals that can conflict. LPs want stale-price value recovered, but traders and solvers need reliable execution. A hook that simply charges very high stale-flow fees can look good in LP accounting while failing to clear. An auction that triggers too often can add complexity where the base hook already works. The proposed design is therefore a selective execution layer around the fee formula, not a replacement for the fee formula.

The 10 bps trigger should be read as a deployment hypothesis. In the October grid, a 5 bps trigger opens more auctions on marginal stale states. Those states still often beat the legacy fixed-fee reference, but they clear less robustly. The 10 bps trigger is more selective and maintains clear rate across the four tested pools.

The solver economics remain unresolved. The October calibration pays solvers about \$1.74 per filled auction on average before gas, with large variation across pools. That is enough to motivate an auction design, but not enough to prove a live solver market. Practical deployment needs batching, low-cost settlement, explicit gas modeling, and multiple-solvers tests.

10 Limitations

The study is a replay, not a live-market experiment. It omits realistic gas costs, competitive solver dynamics, and full routing response. The observed-flow replay adds historical swaps and hook-policy ablations, but it still does not decide whether each historical user swap would route to this pool, another v4 hook pool, a v3 pool, a centralized exchange, or no venue under the changed state. The October grid tests correction trades rather than a full pending-auction state machine. Chainlink is the primary reference oracle, with Pyth, Binance, and deep-pool references reserved for sensitivity checks. The grid does not sweep `alphaBps`. The liquidity-width guard is derived and implemented, but the auction grid does not test LP repositioning behavior or range-width effects on realized order flow.

11 Future Work

The next empirical step is a multi-month replay using the same hook-level benchmark hierarchy: base hook, broad all-stale auction, selective auction, and legacy fixed-fee reference. The next market-design step is a two-pool competitive-routing simulation where traders choose between a base hook pool and a selective-auction hook pool. The auction model should also add gas costs, solver profit margins, multiple solvers, oracle disagreement, batching, and explicit fallback behavior when no solver clears.

12 Conclusion

The proposed design is best understood as a selective v4 hook improvement rather than as a comparison to fixed-fee v3. The base hook prices stale repricing flow with a square-root LVR fee. The Dutch auction adds an execution path only when that path improves LP accounting relative to the base hook. On the observed-flow replay, the selective rule improves LP uplift versus the broad all-stale auction in 19 of 54 windows, leaves 35 unchanged, and worsens 0, while reducing the mean window trigger rate from 7.17% to 3.07%. The fixed-fee v3 comparison remains useful as a leakage check. Stress-regime evidence comes only from the separately measured October corpus; the observed-flow ablation does not contain a measurable stress window.

13 Appendix: Cross-Pool Calibration Check

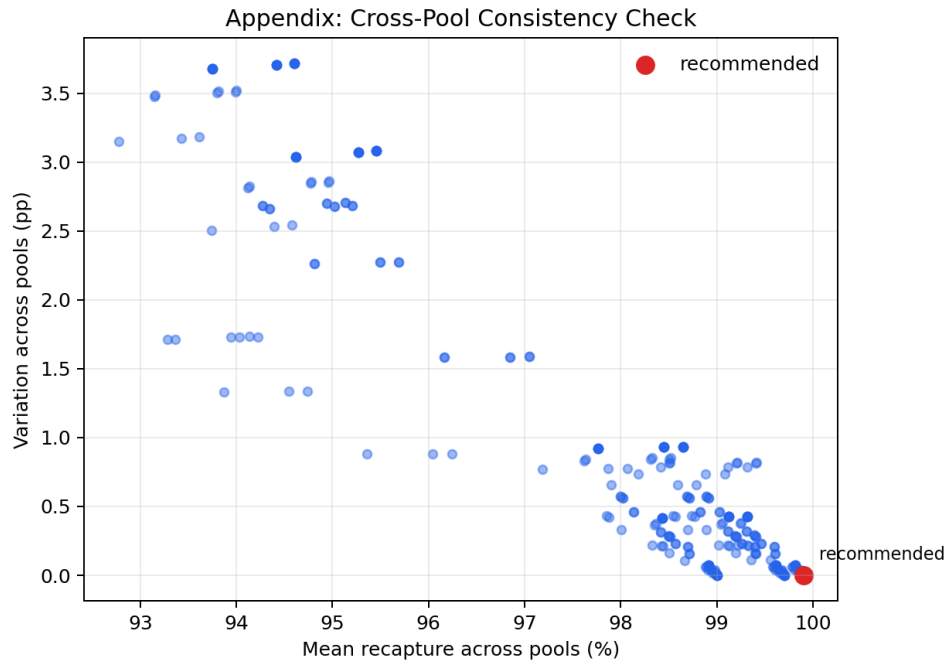


Figure 3: Appendix check: cross-pool mean recapture versus variation across pools for every October 2025 parameter set.

Code and Data Availability

The Solidity hook implementation, replay engine, October 2025 calibration grid, and the figure-generation scripts are released as the open-source repository github.com/mmarfinetz/uni-v4-hook. The empirical results in this paper correspond to evidence release 2026-08-03, frozen under `study_artifacts/evidence_release_2026_08_03`. Its manifest records the source revision and SHA-256 hashes for the replay code and inputs, while CI checks the paper macros and public claims against that release. Figures are regenerated from the frozen artifacts by the scripts under `script/`.

Acknowledgments

The author thanks colleagues who reviewed earlier drafts for feedback on positioning and on the auction-design framing.

References

1. Jason Milionis, Ciamac C. Moallemi, Tim Roughgarden, and Anthony Lee Zhang. “Automated Market Making and Loss-Versus-Rebalancing.” arXiv:2208.06046, August 2022, revised May 2024. DOI: 10.48550/arXiv.2208.06046.
2. Hayden Adams, Noah Zinsmeister, Moody Salem, River Keefer, and Dan Robinson. “Uniswap v3 Core.” Uniswap Labs whitepaper, March 2021. uniswap.org/whitepaper-v3.pdf.
3. Hayden Adams, Moody Salem, Noah Zinsmeister, Sara Reynolds, Austin Adams, Will Pote, Mark Toda, Alice Henshaw, Emily Williams, and Dan Robinson. “Uniswap v4 Core.” Uniswap Labs whitepaper, August 2024. app.uniswap.org/whitepaper-v4.pdf.
4. Uniswap Docs. “Hooks.” Uniswap v4 hooks documentation.
5. Uniswap Docs. “Dynamic Fees.” Uniswap v4 dynamic-fees documentation.
6. Uniswap Labs. “Uniswap v4 TWAMM Hook.” August 24, 2023. Uniswap Labs blog.
7. Uniswap Labs. “Uniswap v4 Truncated Oracle Hook.” September 20, 2023. Uniswap Labs blog.
8. Austin Adams, Ciamac C. Moallemi, Sara Reynolds, and Dan Robinson. “am-AMM: An Auction-Managed Automated Market Maker.” arXiv:2403.03367, March 2024.
9. Eric Budish, Peter Cramton, and John Shim. “The High-Frequency Trading Arms Race: Frequent Batch Auctions as a Market Design Response.” *The Quarterly Journal of Economics*, 130(4):1547–1621, 2015. DOI: 10.1093/qje/qjv027.